

TF-IDF & IDF, Worked Out by Hand

Why a word that appears in every document tells you nothing, and a rare word tells you almost everything — the logarithmic inverse-document-frequency weight that underlies all of keyword search, derived and computed on a ten-document corpus.

What this document does. It builds the two halves of a keyword relevance score — *term frequency* (how often a word occurs in a document) and *inverse document frequency* (how rare the word is across the corpus) — and computes IDF by hand for a common, a medium, and a rare term. It explains why the log is there, and connects the classic IDF to the BM25 variant used in PDF #15. Numbers from Appendix A.

Leads into: the BM25 formula (#15) and its k_1, b knobs (#16), which refine exactly these two ingredients.

1 Two ingredients of keyword relevance

A document is relevant to a query term for two reasons that pull in opposite directions:

- **Term frequency** $\text{tf}(t, d)$: the more a document uses the term, the more it is “about” it. Pushes the score *up*.
- **Document frequency** $\text{df}(t)$: the more documents contain the term, the less it distinguishes any one of them. Should push the score *down*.

TF-IDF multiplies a rising function of the first by a *falling* function of the second:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \text{idf}(t), \quad \text{idf}(t) = \ln \frac{N}{\text{df}(t)}$$

where N is the number of documents in the corpus.

Intuition

IDF is a surprise meter. “The” appears in every document, so finding it surprises you not at all — it carries zero information about which document you want. A word like “quokka” appears in one document in ten thousand, so seeing it almost *names* the document. IDF turns “how rare” into “how informative,” and the rarer the word, the louder its vote. TF then says: among documents that use this informative word, prefer the ones that use it a lot.

2 Why the logarithm?

Rarity spans orders of magnitude: a term might appear in 1 document or in 10,000. A raw ratio N/df would make the rarest term thousands of times more important than a merely uncommon one — wildly over-weighting hapax words and letting a single exotic match dominate everything. The log *compresses* this range, so each $10\times$ drop in document frequency adds a roughly constant bump to the weight, not a multiplicative explosion:

$$\text{idf} = \ln \frac{N}{\text{df}} \Rightarrow \text{df} \rightarrow \text{df}/10 \text{ adds } \ln 10 \approx 2.30 \text{ to idf, regardless of scale.}$$

The log turns “rarity” (a multiplicative quantity) into “information” (an additive one) — the same reason information content is measured in log units (bits, nats).

3 Worked by hand on a 10-document corpus

$N = 10$. Three query terms with different document frequencies:

term	df	idf = $\ln(N/df)$	reading
the	10	$\ln(10/10) = \ln 1 = 0.0000$	in every doc: useless
network	3	$\ln(10/3) = 1.2040$	moderately selective
quokka	1	$\ln(10/1) = 2.3026$	nearly a unique key

the gets weight *exactly zero* — it cannot help rank anything, which is why stop-words contribute nothing even if you never remove them. **quokka**, in one document, gets the largest weight: a single occurrence is strong evidence.

Mini-example · TF and IDF together

A document uses **network** 3 times and **the** 12 times. Their contributions to the TF-IDF score of a query "**the network**":

$$\text{the} : 12 \times 0.0000 = 0, \quad \text{network} : 3 \times 1.2040 = 3.612.$$

The document mentions **the** four times as often as **network**, yet **network** supplies 100% of the score. Frequency without selectivity is worth nothing; IDF is what makes keyword search ignore the filler.

4 The BM25 variant of IDF

Modern keyword search (BM25, PDF #15) uses a smoothed, “probabilistic” IDF that never goes negative and is gentle on common terms:

$$\text{idf}_{\text{BM25}}(t) = \ln \left(\frac{N - \text{df}(t) + 0.5}{\text{df}(t) + 0.5} + 1 \right).$$

The +0.5 terms smooth tiny counts; the +1 inside the log keeps it positive. On the same corpus:

term	df	idf _{classic}	idf _{BM25}
the	10	0.0000	0.0465
network	3	1.2040	1.1451
quokka	1	2.3026	1.9924

Same story — rare beats common — just regularised so a stop-word earns a hair above zero instead of exactly zero, and the rarest term is not over-rewarded. This is the idf that walks into the BM25 score next.

Equation cheat-sheet

$$\begin{aligned} \text{tfidf}(t, d) &= \text{tf}(t, d) \cdot \text{idf}(t), & \text{idf}(t) &= \ln \frac{N}{\text{df}(t)} \\ \text{df} \uparrow &\Rightarrow \text{idf} \downarrow, & \text{idf}_{\text{BM25}} &= \ln \left(\frac{N - \text{df} + 0.5}{\text{df} + 0.5} + 1 \right) \geq 0 \end{aligned}$$

Appendix A · Reference implementation

```
import numpy as np
N = 10
def idf_classic(df): return np.log(N / df)
```

```
def idf_bm25(df):    return np.log((N - df + 0.5) / (df + 0.5) + 1)

for df in [10, 3, 1]:    # the, network, quokka
    print(df, round(idf_classic(df), 4), round(idf_bm25(df), 4))
# 10 -> 0.0000 0.0465    3 -> 1.2040 1.1451    1 -> 2.3026 1.9924
```

Internal learning material. Numbers reproducible from the appendix code. v1.0